Protobuf Dynamic Transcoder — Stage 2 Spec: Proto2 + Editions 2024

Prerequisites

Stage 1 (MVP) is complete. We have a working proto3 decode pipeline: `.proto` parser → descriptor model → CodedReader → DynamicMessage → JSON.

Goal

Extend the library to handle proto2 and Editions 2024 `.proto` files. After this stage, the library decodes any valid protobuf message regardless of which syntax/edition it was defined with.

What Changes Per Layer

Proto Parser

Proto2 additions (see Proto2 Language Guide):

- `syntax = "proto2";`
- `required` label
- `optional` label (explicit — different semantics from proto3 `optional`)
- `group` field type: `optional group MyGroup = 1 { ... }` — defines both a field and a nested message in one declaration
- `extensions` range declaration: `extensions 100 to 199;`
- `extend` blocks: fields defined outside the message they belong to
- `default` field option: `optional int32 x = 1 [default = 42];`
- `packed` is opt-in (`[packed = true]`) instead of default

Editions 2024 additions (see Editions Language Guide):

- `edition = "2024";` instead of `syntax`

- `features` block at file, message, field, enum, and oneof level:

```
edition = "2024";
option features.field_presence = EXPLICIT;

message Foo {
  int32 x = 1 [features.field_presence = IMPLICIT];
}
```

- Feature keys to parse: `field_presence`, `enum_type`, `repeated_field_encoding`, `message_encoding`, `json_format`
- Features are defined in `descriptor.proto` as the `FeatureSet` message

Parser impact: Moderate. The grammar extensions are well-defined. `group` is the most awkward — it simultaneously declares a field and a nested message type. `extend` blocks require the parser to attach fields to a message defined elsewhere. Default value parsing needs to handle all scalar literal types (integers, floats, string literals, enum value names, `true`/`false`).

Descriptor Model

This is the largest area of change. Four new concerns.

1. Field Presence

Proto3 implicit presence: a scalar field has no "set" vs "unset" distinction. The value is either the default (zero/empty/false) or not.

Proto2 explicit presence: every singular field tracks whether it was seen in the wire data, independent of its value. A field set to its default value is still "present."

Editions: controlled by `features.field_presence` (defined in `descriptor.proto` FeatureSet):

- `IMPLICIT` → proto3 behavior
- `EXPLICIT` → proto2 behavior
- `LEGACY_REQUIRED` → proto2 `required` behavior

Java's `FieldDescriptor` handles this via `hasPresence()` and `isRequired()` in `Descriptors.java` — search for `hasPresence` and `getFeatures().getFieldPresence()`.

Changes to `FieldDescriptor`:

```
public sealed class FieldDescriptor
{
    // ... existing properties from MVP ...

    public FieldPresence Presence { get; }
}

public enum FieldPresence
{
    Implicit,
    Explicit,
    Required,
}
```

For proto3: all singular scalar fields are `Implicit`. Fields inside a `oneof` or `optional` (proto3 explicit optional) are `Explicit`. Message-typed fields are always `Explicit`.

For proto2: `optional` and `required` fields are `Explicit` / `Required`. `repeated` fields don't have presence.

For editions: resolved from `features.field_presence` with inheritance (file → message → field).

2. Default Values

Proto2 allows per-field default values. Proto3 defaults are always zero-value. Editions inherit from the syntax they're emulating.

Java handles this in `FieldDescriptor.getDefaultValue()` and `FieldDescriptor.hasDefaultValue()` in `Descriptors.java`. `DynamicMessage.getField()` returns the default when a field isn't set — see `DynamicMessage.java` `getField()` method.

Changes to `FieldDescriptor`:

```
public sealed class FieldDescriptor
{
    // ... existing properties ...

    public object? DefaultValue { get; }
    public bool HasExplicitDefault { get; }
}
```

Default value types:

Field type	Default if not specified	Example explicit default

int32/sint32/sfixed32	`0`	`[default = 42]`
int64/sint64/sfixed64	`0L`	`[default = -1]`
uint32/fixed32	`0u`	`[default = 100]`
uint64/fixed64	`0uL`	
float	`0.0f`	`[default = 3.14]`
double	`0.0`	`[default = 2.718]`
bool	`false`	`[default = true]`
string	`""`	`[default = "hello"]`
bytes	`byte[0]`	`[default = "\x01\x02"]`
enum	first value in enum definition	`[default = BAR]`

3. Groups

Groups (proto2 only, or editions with `message_encoding = DELIMITED`) use wire types 3 (StartGroup) and 4 (EndGroup) instead of wire type 2 (LengthDelimited) for embedded messages. See the Encoding Reference on groups (search for "SGROUP"/"EGROUP").

Java handles group reading in `CodedInputStream.java` — `readGroup()` method and related `readTag()` / `checkLastTagWas()` for EndGroup validation. The C++ equivalent is cleaner: see `wire_format.cc` `ParseAndMergePartial()` where it handles `WIRETYPE_START_GROUP` by recursing into `readGroup()`.

Changes to `FieldDescriptor`:

```
public sealed class FieldDescriptor
{
    // ... existing properties ...

    public bool IsGroupEncoded { get; }
}
```

The parser desugars `group` declarations into a nested message type + a field referencing it, with `IsGroupEncoded = true`.

4. Extensions

Extensions are fields defined in `extend` blocks that attach to a message declared elsewhere. They have field numbers from the message's `extensions` range.

Java handles this with [ExtensionRegistry.java] — a table of known extensions searchable by message type + field number. During decode, if a field number isn't found in the message's own fields, the extension registry is consulted. Unknown extensions are treated as unknown fields (skipped or preserved in `UnknownFieldSet`).

The extension ranges are declared in the `DescriptorProto.extension_range` field in [descriptor.proto].

```
public sealed class ExtensionDescriptor
{
    public FieldDescriptor Field { get; }
    public MessageDescriptor ExtendeeType { get; }
}

public sealed class MessageDescriptor
{
    // ... existing properties ...

    public IReadOnlyList<ExtensionRange> ExtensionRanges { get; }

    // Already exists from MVP. Now also checks the extension registry
    // when the number falls within an extension range.
    public FieldDescriptor? FindFieldByNumber(int number);
}

public readonly record struct ExtensionRange(int Start, int End);
```

The `DescriptorPool` collects all `extend` blocks from all loaded files and builds an extension registry:

```
public sealed class DescriptorPool
{
    // ... existing ...

    internal FieldDescriptor? FindExtension(string messageFullName, int fieldNumbe
}
```

`MessageDescriptor.FindFieldByNumber()` checks own fields first, then calls `pool.FindExtension()`. This is transparent to the decode loop.

5. Editions Feature Resolution

Features are inherited: file → message → field (and file → enum, message → oneof → field, etc.). Child features override parent features.

The feature definitions and their defaults are in the `FeatureSet` message in `descriptor.proto`. Java resolves features in `Descriptors.java` — search for `resolveFeatures` and `getFeatures()`.

```
internal sealed class FeatureSet
{
    public FieldPresence FieldPresence { get; init; }
    public EnumType EnumType { get; init; }
    public RepeatedFieldEncoding RepeatedFieldEncoding { get; init; }
    public MessageEncoding MessageEncoding { get; init; }
    public JsonFormat JsonFormat { get; init; }

    public static FeatureSet Resolve(FeatureSet parent, FeatureSet? overrides);
}

internal enum EnumType { Open, Closed }
internal enum RepeatedFieldEncoding { Packed, Expanded }
internal enum MessageEncoding { LengthPrefixed, Delimited }
internal enum JsonFormat { Allow, LegacyBestEffort }
```

Default feature sets per syntax:

Feature	Proto2 default	Proto3 default	Editions 2024 default
field_presence	EXPLICIT	IMPLICIT	EXPLICIT
enum_type	CLOSED	OPEN	OPEN
repeated_field_encoding	EXPANDED	PACKED	PACKED
message_encoding	LENGTH_PREFIXED	LENGTH_PREFIXED	LENGTH_PREFIXED
json_format	LEGACY_BEST_EFFORT	ALLOW	ALLOW

The resolved features are baked into the descriptors (e.g., `FieldDescriptor.Presence` is derived from the resolved `field_presence` feature).

CodedReader

One addition: **group decoding.**

```
public ref struct CodedReader
{
    // ... existing from MVP ...

    public CodedReader ReadGroup(int fieldNumber);
}
```

Implementation: read fields until we encounter the EndGroup tag matching the given field number. Return a sub-reader over the bytes between StartGroup and EndGroup.

This is the only wire format change.

DynamicMessage

Presence tracking: Add a bitset alongside the field values. Set the bit when a field is decoded from the wire data.

```
public sealed class DynamicMessage
{
    // ... existing from MVP ...

    public bool HasField(FieldDescriptor field);
    // ↑ Already in MVP, but now backed by a real presence bitset
    // instead of just checking for non-null values.
}
```

Decode loop change for groups:

```
ReadValue(reader, field):
    match field.Type:
        MESSAGE when field.IsGroupEncoded →
            DynamicMessage.Decode(field.MessageType, reader.ReadGroup(field.Number
        MESSAGE →
            DynamicMessage.Decode(field.MessageType, reader.ReadMessage())
        // ... rest unchanged
```

Decode loop change for extensions: None. `FindFieldByNumber` handles it transparently.

JSON Serialization

Presence-aware emission:

```
WriteMessage(message, writer):
    for each field in message.Descriptor.Fields (+ extensions):
        if field.Presence == Implicit:
```

```
        value = message.GetField(field)
        if not EmitDefaults and value equals field.DefaultValue:
            skip
    else if field.Presence == Explicit or Required:
        if not EmitDefaults and not message.HasField(field):
            skip

    writeField(field, value)
```

Closed enums (proto2 / editions with `enum_type = CLOSED`): If the decoded enum number has no matching `EnumValueDescriptor`, proto2 behavior puts it in unknown fields. For JSON output, skip it or emit the integer (configurable).

Required field validation: Optional. If enabled, after decoding, check that all `Required`-presence fields have `HasField() == true`. Default: log but don't throw (matches most implementations' lenient behavior).

Summary of Changes by Component

Component	Change	Size
Proto parser	`proto2` syntax, `group`, `extend`, `extensions`, `default`, `required`	Medium
Proto parser	`edition = "2024"`, `features` blocks	Small-Medium
Descriptor model	`FieldPresence` enum, `DefaultValue`, `HasExplicitDefault`	Small
Descriptor model	`IsGroupEncoded` on `FieldDescriptor`	Small
Descriptor model	`ExtensionDescriptor`, extension registry, extension lookup in `FindFieldByNumber`	Medium
Descriptor model	Feature resolution + inheritance	Medium
CodedReader	`ReadGroup(fieldNumber)`	Small
DynamicMessage	Presence bitset + `HasField()` semantics	Small
DynamicMessage decode	Group decoding branch	Small

JSON serializer	Presence-aware field emission, closed enum handling	Small-Medium

---

Implementation Order

1. **Feature resolution engine** — `FeatureSet` with defaults per syntax and merge logic. Unit test: given a proto2 file, resolve features to proto2 defaults. Given an editions file with overrides, resolve correctly.
2. **Proto parser: proto2** — Add `required`, `optional`, `group`, `extensions`, `extend`, `default`. Test by parsing `descriptor.proto` itself (it's proto2) and other proto2 well-known files.
3. **Proto parser: editions** — Add `edition` keyword, `features` blocks. Test with editions versions of standard protos from the protobuf conformance test suite.
4. **Descriptor model: presence + defaults** — `FieldPresence`, `DefaultValue`, wire up from parser output + resolved features.
5. **Descriptor model: extensions** — `ExtensionDescriptor`, extension registry, `FindFieldByNumber` extension fallback. Reference: `ExtensionRegistry.java`.
6. **CodedReader: groups** — `ReadGroup`. Test against protobuf bytes containing group-encoded messages.
7. **DynamicMessage: presence tracking** — Bitset, `HasField()`.
8. **JSON serializer: presence-aware emission** — Update field emission logic.
9. **Integration** — End-to-end tests: proto2 `.proto` file + proto2-encoded bytes → decode → JSON. Same for editions 2024.

Out of Scope (for Stage 2)

- Encode (JSON → protobuf, DynamicMessage → bytes) — per PRD, not building this
- `MessageSet` wire format (a Google-internal proto2 extension encoding; extremely rare outside Google)
- Proto2 `services` semantics
- Custom option interpretation (options are parsed and stored but not semantically evaluated beyond `default`, `packed`, and `features`)

References

Official Protobuf Specifications

- Proto2 Language Guide — required/optional, extensions, groups, default values
- Editions Language Guide — edition syntax, features blocks, migration from proto2/proto3
- Wire Format Encoding Reference — group wire types (SGROUP/EGROUP), packed repeated fields
- ProtoJSON Format — JSON mapping, presence semantics in JSON
- `descriptor.proto` — `FeatureSet` message, `ExtensionRange`, `FieldDescriptorProto.default_value`

Java Source Code (port reference for Stage 2 additions)

- `Descriptors.java` — `hasPresence()`, `isRequired()`, `getDefaultValue()`, `resolveFeatures()`, extension range handling
- `DynamicMessage.java` — `getField()` default value fallback, `isInitialized()` required field check
- `ExtensionRegistry.java` — extension lookup by containing type + field number
- `CodedInputStream.java` — `readGroup()` method, EndGroup tag validation
- `JsonFormat.java` — `Printer.print()` field emission logic, `includingDefaultValueFields` option

C++ Source Code (supplementary)

- `wire_format.cc` — `ParseAndMergePartial()` handling of `WIRETYPE_START_GROUP`, extension lookup via `FindKnownExtensionByNumber()`
- `dynamic_message.h` — `DynamicMessageFactory` pattern

Test Resources

- Protobuf conformance test suite — canonical test vectors for JSON mapping, proto2/proto3/editions behavior
- `descriptor.proto` — a complex proto2 file, good parser validation target